Enhanced Android Malware Detection Using LightGBM with Optimized Hyperparameters, RobustScaler Normalization, and SMOTE on CICMalDroid2020 Dataset

Masayu Octa Faradisa
Faculty of Computer Science
Universitas Dian Nuswantoro
Semarang, Indonesia
111202315374@mhs.dinus.ac.id

Wildanil Ghozi
Faculty of Computer Science
Universitas Dian Nuswantoro
Semarang, Indonesia
wildanil.ghozi@dsn.dinus.ac.id

Ramadhan Rakhmat Sani
Department of Information System
Universitas Dian Nuswantoro
Semarang, Indonesia
ramadhan_rs@dsn.dinus.ac.id

Nor Hafeizah Hassan
Faculty of Information Communication and Technology
Universiti Teknikal Malaysia
Melaka, Malaysia
nor_hafeizah@utem.edu.my

Ibrahim Budak
Data Analysis Monitoring and Evaluation Office
Kastamonu University
Kastamonu, Türkiye
ibudak@kastamonu.edu.tr

Fauzi Adi Rafrastara
Faculty of Computer Science
Universitas Dian Nuswantoro
Semarang, Indonesia
fauziadi@dsn.dinus.ac.id

***Abstract*— Android malware detection remains a critical challenge due to the increasing sophistication of malicious applications. Existing detection approaches often rely on normalization techniques sensitive to outliers, imbalance-handling strategies prone to data loss or leakage, and single hold-out evaluation schemes that can produce inconsistent performance estimates. This study proposes an enhanced detection pipeline using LightGBM with optimized hyperparameters, Log1p and RobustScaler normalization, and in-fold SMOTE oversampling on the CICMalDroid2020 dataset. Preprocessing removed 5,815 duplicate samples (33.5%) from the original 17,341 records to prevent artificially inflated accuracy, while Log1p and RobustScaler compressed outliers and stabilized feature distributions without relying on mean-based statistics. Class imbalance was addressed through SMOTE applied strictly within each training fold to avoid data leakage. Several tree-based and ensemble models were compared, with LightGBM achieving the best results: 96.36% accuracy on the 80:20 hold-out split and 95.58% accuracy under 5-Fold Cross Validation, confirming strong consistency across folds. These results outperform previously reported benchmark methods on the same dataset by up to 3.08%. The proposed pipeline demonstrates that combining robust normalization, leakage-free oversampling, and rigorous cross-validation improves the reliability and generalizability of Android malware classification, offering a practical, reproducible reference pipeline for Android malware classification.**

Keywords—Android Malware, LightGBM, SMOTE, RobustScaler, CICMalDroid2020.

I. INTRODUCTION

Malware is one of the most destructive weapons in the digital space [1]. There are many types of malware. Adware, Ransomware, SMS malware, and Scareware are few examples [2]. Each type of malware causes different kinds of damage.

Nowadays, more smartphones and operating systems are being developed. In fact, people are spending more time on their smartphones [3], [4]. This is evidenced by 95% of teenagers in America and Europe spend their time on smartphones and avoiding direct face-to-face interactions [5]. In Switzerland, 99% of children aged 12 – 19 spend 4 – 5 hours per day on their smartphones playing online games and additionally, adults spend their time on mobile applications such as social media [5]. This makes them a target for cybersecurity attacks. Malware often targets smartphones and similar devices, including the applications on them.

Android has been becoming the dominant player in the smartphone market, with 7.62% in 2013 [6] until 70.1% in 2023 [7]. As a result, cybersecurity criminals are spreading malware on a large scale, and some are developing malware using high-level programming languages [8]. According to the report published in 2020, 16,000 new malware samples were detected every day on Google Play Store, a place where Android users download apps [2]. Consequently, many people fall victim to this malware.

Besides damaging systems and causing data theft, malware can also result in financial losses [7]. As we know, one of them is Ransomware which works by locking data and the victim has to pay a nominal amount of money [2]. Mehrban et al. [10] note that ransomware attacks have increased substantially in recent years, with organizations across government, private companies, and individual users suffering lasting damage including data loss, operational disruption, and persistent vulnerabilities.f In this case, the resulting losses in the form of bitcoin. For example, Beazly an insurance company, paid 42 bitcoins to ransom data that was affected by malware [11].

In carrying out all these preventive measures, many security researchers are continuously developing strategies to prevent malware attacks. Some examples like traditional methods, include static and dynamic analysis methods, however these methods are still not fully effective against modern, polymorphic malware [12]. Therefore, the experiment in this study was conducted using Machine Learning (ML), as there are numerous real-world results that could potentially be more accurate in malware prevention [9], [12].

Despite the progress in ML-based detection, several critical research gaps remain in recent studies. Previous State-Of-The-Arts (SOTA) methods, such as those by Sabrina et al.

[13] and Ansori et al. [14], achieved notable results on the CICMalDroid2020 dataset but relied on simple normalization techniques (e.g., MinMax or Standard Scaler) that are highly sensitive to outliers inherent in system call features. Furthermore, class imbalance is often handled via under-sampling or without strict in-fold application during cross-validation, which can lead to data leakage and overly optimistic performance estimates. Finally, the reliance on single hold-out validation lacks the robustness required to guarantee model generalizability.

To address these gaps, this research does not propose a new algorithm, but instead presents a systematic integration of established preprocessing and modelling techniques into a single, leakage-free Android malware detection pipeline. The novelty of this study lies in how robust normalization, in-fold oversampling, and rigorous cross-validation are combined and empirically validated together, rather than in any individual component. The main contributions of this integration are as follows:

- A preprocessing pipeline contains robust normalization technique combining Log1p transformation and RobustScaler to effectively handle outliers in statistical features, resulting in higher accuracy than previous State-Of-The-Arts [13], [14] methods.
- Applying stricter validation using 5-Fold Cross Validation with in-fold SMOTE oversampling ensure more accurate results and maintain consistency with the repeated training, also prevents data leakage between training and testing folds.
- Comparing optimized tree-based models such as LightGBM, XGBoost, RandomForest, ExtraTrees, and various other combinations in order to identify the most effective architecture for Android malware classification.

Based on these contributions, there is also a process carried out to ensure those contributions can be accurately realized by combining three main components,

- Data normalization using Log1p and RobustScaler to compress outliers and stabilize feature distributions.
- SMOTE oversampling applied in-fold to prevent data leakage.
- Hyperparameter optimization and validation using 5-Fold Cross Validation to ensure the generalizability of the final model.

In summary, the objective of this study is to develop and empirically validate a leakage-free, outlier-robust Android malware detection pipeline based on optimized LightGBM that achieves higher accuracy and F1-Score that existing State-Of-The-Arts methods on the CICMalDroid2020 dataset, while ensuring the reliability and generalizability of the results through 5-Fold Cross Validation.

II. RELATED WORK

A. Android Malware Detection Using Machine Learning

This research refers to 2 other researches, namely the research by Sabrina et al. [13] and Ansori et al. [14]. As can be seen from Table I, these researches have different methods, but both are similar in that they minimize their features.

TABLE I. STATE-OF-THE-ARTS

Researcher	Method	Dataset	Accuracy
Sabrina et al. (2024)	RF + FCBF + 10-fold CV	CICMalDroid2020	92.5%
Ansori et al. (2024)	Gain Ratio + Ens RF+ET+kNN	CICMalDroid2020	94.57%

B. Statistical Feature-Based Research

Sabrina et al. [13] using Symmetrical Uncertainty method, which correlates features with a table and selects as few features while achieving the highest possible accuracy. This research used 48 features out of 470 features and resulting an accuracy of 92.50% using RandomForest model.

In this research, to handling the class imbalance is using under-sampling, which reduces the majority class instances to match the minority class. Under-sampling inevitably discards potentially valuable information from the majority class, leading to suboptimal model generalization. Meanwhile this studies also using MinMax Normalization, which mean that method is highly sensitive to extreme outliers.

C. Ensamble-Based Research

Ansori et al. [14] used a combine method of three models: RandomForest, ExtraTrees, and k-Nearest Neighbor (kNN) and applied hard voting on CICMalDroid2020 dataset. Additionally, gain ratio was also applied and resulting in only 171 features out of 470 features. This research achieved the highest accuracy of 94.57% and F1-Score of 94.58%.

In this research, using StandardScaler for the normalization. That means also highly sensitive to extreme outliers like MinMax Normalization in Sabrina et al. [13] methods. Besides that, this research relied on a single hold out validation and this approach is prone to split bias, where the model's performance may be artificially inflated or deflated depending on the specific data partition.

D. Summary of Related Works and Reseacrh Gap

Table II summarizes the methodological differences between the proposed approach and the State-Of-The-Arts (SOTA) studies.

Based on the review above, several gaps have been identified:

TABLE II. COMPARISON OF METHODOLOGIES

Aspect	Sabrina et al. [13]	Ansori et al. [14]	Proposed Method
Imbalance Handling	Under-sampling (Data loss)	Not explicitly stated	In-Fold SMOTE (No data loss/ leakage) [15]
Normalization	MinMax (Sensitive to outliers)	StandardScaler (Sensitive to outliers)	Log1p + RObustScaler (Outliers resilient)
Feature Selection	FCBF (48 features)	Gain Ratio (171 features)	Full Statistical Features (470 features)
Primary Models	Random Forest	RF, ET, k-NN, SVM, NB	LightGBM, XGBoost, RF, ET
Validation Strategy	10-Fold CV	Hold-out (80:20)	5-Fold Cross Validation

Based on the previous studies by Sabrina et al. [13] achieved 92.5% accuracy using Random Forest with FCBF feature selection, while Ansori et al. [14] reached 94.57%

using ensemble learning with Gain Ratio. However, both studies relied on single hold-out validation which may lead to overfitting on specific data splits. This research addresses this limitation by implementing 5-Fold Cross Validation with in-fold SMOTE to ensure robust performance estimation and prevent data leakage [16].

III. METHODOLOGY

In this research, a pipeline is used to explain how the experiment progressed from start to finish, in order to achieving accuracy that exceeds the State-Of-The-Arts [13], [14]. Several stages were developed by added new methods from previous researches. Starting from the dataset used for the experiment to the final result, can be seen in the Fig. 1.

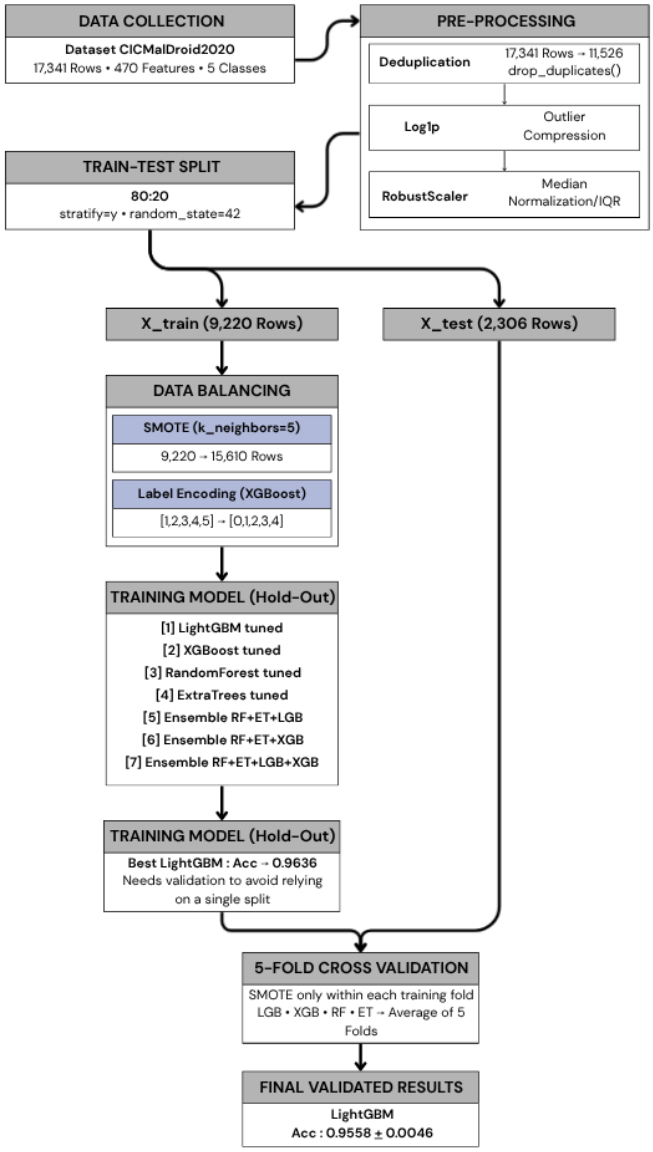


Fig. 1. Overview of the integrated preprocessing and modeling pipeline for Android malware detection

A. Dataset

This research uses the CICMalDroid2020 dataset developed by the Canadian Institute for Cybersecurity [3]. The dataset consists of 17,341 Android application samples divided into five categories: Benign (2,000 samples), Adware (4, 354 samples), Banking Malware (1,713 samples), SMS Malware (1,760 samples), and Riskware (7,514 samples). Each sample is represented by 470 statistical features extracted from dynamic analysis results using Android system calls. As noted by Abedin et al. [3] and Ansori et al. [14], the dataset exhibits a significant class imbalance. This inherent imbalance distribution is the main challenge and can be addressed using the SMOTE technique.

B. Preprocessing

In this CICMalDroid2020 dataset, there are several preprocessing steps that need to be carried out. Data preprocessing is a critical of tree-based and ensemble models.

1. Deduplication

Deduplication is critical in malware detection as duplicate sample can lead to overfitting and inflated performance metrics. Sample duplication in malware datasets has been shown to artificially increase accuracy by up to 15% if not properly handled [17].

This duplicate sample removal used drop_duplicates() to ensure that every sample in the dataset is unique, eliminating bias due to duplicate data, so it doesn't affect in the training and evaluation processes. Following the best practices for cybersecurity datasets recommended by Mu et al. [18] the number of samples after removal became 11,526 from 17,341 samples.

2. Log1p Transformation

Log1p transformation is applied to handle outliers and reduce skewness in feature distributions especially for the CICMalDroid2020 dataset. This method is mathematically define as, $x' = \ln(1 + |x|)$ to absolute values of the features. Unlike standard log transformation, log1p can handle zero values without errors, making it suitable for sparse malware feature data without introducing computational errors [19]

By compressing the dynamic range of extreme outliers, Log1p significantly reduces feature skewness and stabilizes variance [20]. This transformation is crucial prior to scaling, ultimately leading to more symmetric feature distributions and improved model generalization [21].

3. RobustScaler Normalization

The final preprocessing step is normalization using RobustScaler, which is applied after Log1p because normalization is based on the median and IQR, rather than the mean and standard deviation. RobustScaler was chosen because it is resistant to extreme outliers that usually found in system call features and malware datasets, ensuring that the feature distributions are stabilized without compressing the variance of the normal data points [22].

C. Handling Class Imbalance

Class imbalance in the dataset must be handled to balance the data, so it does not affect the final results. SMOTE is applied in this research to address the imbalance by generating new synthetic data for the minority classes using k_neighbors=5 parameter.

SMOTE is applied to the training data in each fold to prevent data leakage that could lead to over perfection of performance estimates. Fig. 2 illustrates the significant shift in class distribution before and after the SMOTE oversampling process, ensuring that the model does not bias towards the majority class.

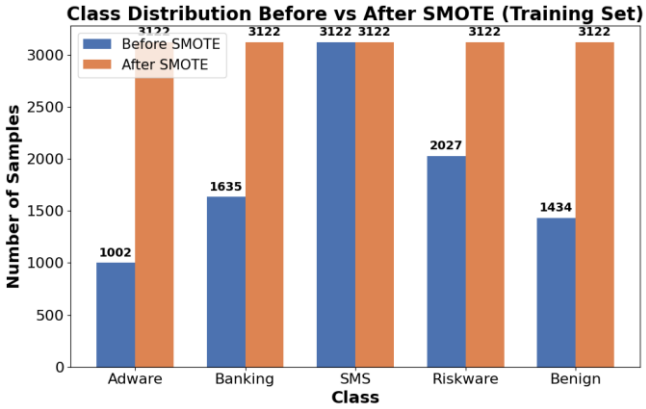


Fig. 2. Class distribution comparison before and after applying SMOTE

Additionally, label encoding is also applied to adjust the class labels for the XGBoost model, which requires labels to start from 0. The original labels [1, 2, 3, 4, 5] are remapped to [0, 1, 2, 3, 4] to meet this requirement.

D. Split Data

After the dataset goes through the preprocessing stage, it is continued by splitting the dataset into two with a ratio of 80:20, which is called the holdout method. The split is performed using the stratify=y parameter to ensure that the proportion of each class is evenly maintained, because the class distribution is unbalanced. The random_state=42 parameter is used to maintain the reproducibility of the experimental results [23]. The split results in 9,220 samples (80%) for training data and 2,306 samples (20%) for testing data before the oversampling process is applied.

E. Models and Hyperparameters

Based on the purpose and motivation of this research, several other models are used to compare the final performance results, including the LightGBM model as the primary model. LightGBM was chosen as the primary model due to its efficiency in handling high-dimensional data [24]. The hyperparameters used are, n_estimators=1000, max_depth=8, learning_rate=0.3, num_leaves=127, subsample=0.8, colsample_bytree=0.7, min_child_samples= 20, reg_alpha=0.5, reg_lambda=0.5, and class_weight= 'balanced'.

The additional models used for comparison and evaluation are, XGBoost with n_estimators=1000 and max_depth=7 was include as a robust gradient-boosting alternative [25], while RandomForest with n_estimators=700 ExtraTrees with n_estimators=700 were utilized in bagging-based ensemble methods (RF+ET+LGB, RF+ET+XGB, RF+ET+LGB+XGB) that reduce variance through bootstrap aggregation and random feature selection [26].

F. Validation

The evaluation is carried out in two stages, the 80:20 holdout scheme and 5-Fold Cross Validation. First, the 80:20 holdout scheme is obtained from the initial using seven model configurations. Meanwhile, the 5-Fold Cross Validation is applied to the four best models (LightGBM, XGBoost, RandomForest, ExtraTrees) using the entire dataset to validate the consistency of the results [27].

For each fold, SMOTE is applied only to the training fold before model training, to prevent data leakage between folds. The evaluation metrics that used are Accuracy, Precision, Recall, and F1-Score [28].

IV. RESULT AND DISCUSSION

After doing all experiments, this research produced some satisfying accuracy results. After the new models used, it shows an improvement in accuracy compared to previous researches.

A. Hold-Out Results (80:20)

Table III presents all models evaluated using the Hold-Out 80:20 split. These results were obtained by applying SMOTE, with a training set size of 15,610 samples from 13,872 samples (across five classes), while the test set remained at 2,306 samples.

TABLE III. Hold-Out Evaluation Results (80:20)

Models	Acc	Pre	Rec	F1
LightGBM	96.36%	96.41%	96.36%	96.36%
XGBoost	96.18%	96.22%	96.18%	96.18%
Ens (RF++ET+LGB+XGB)	95.92%	96.06%	95.92%	95.95%
Ens (RF+ET+LGB)	95.71%	95.86%	95.71%	95.73%
Ens(RF+ET+XGB)	95.71%	95.86%	95.71%	95.73%
ExtraTrees	95.49%	95.68%	95.49%	95.52%
RandomForest	95.45%	95.59%	95.45%	95.47%

LightGBM achieved the best performance among all the models in this Hold-Out evaluation, with an accuracy of 96.36% and followed by XGBoost with an accuracy of 96.18%. all tested models exceeded the best baseline result from Sabrina et al. [13] (92.5%) and Ansori et al. [14] (94.57%) researches. This demonstrates the effectiveness of the proposed preprocessing pipeline, which utilize histogram-based splitting and leaf-wise tree growth to efficiently capture complex, non-linear patterns in high-dimensional malware feature while incorporating built-in regularization to mitigate overfitting [24].

B. 5-Fold Cross Validation Results

Table IV presents the results after performing 5-Fold Cross Validation, the primary evaluation of this research. This validation is important to ensure the consistent results, and the Hold-Out results are not just dependent on a single data split. Table IV details the mean performance across the five folds.

TABLE IV. 5-Fold Cross Validation Results

Models	CV Acc	CV F1	CV Prec
LightGBM	95.58%	95.59%	95.64%
XGBoost	95.51%	95.51%	95.55%
RandomForest	95.01%	95.03%	95.16%
ExtraTrees	94.93%	94.96%	95.09%

Similar to the Hold-Out results, LightGBM was the best performing model with a CV accuracy of 95.58%. the low standard deviation indicates that the model does not overfit to any particular data split. XGBoost also came in second with a CV accuracy of 95.51%. the marginal drop in accuracy from the Hold-Out to the CV evaluation is expected and demonstrates that the Hold-Out results were not artificially inflated by a lucky data split [23].

C. Comparison with State-Of-The-Arts

Table V presents a comparison between the proposed method and two recent studies on the same CICMalDroid2020 dataset.

TABLE V. Comparison With State-Of-The-Arts

Method	Years	Acc	F1
Sabrina et al. [13] (RF+FCBF)	2024	92.50%	92.50%
Ansori et al. [14] (RF+ET+kNN)	2024	94.57%	94.58%
Proposed (LightGBM tuned)	**2025**	**95.58%**	**95.59%**

The proposed method achieves an accuracy of 95.58% and an F1-Score of 95.59% based on 5-Fold CV, proving to outperform all baselines. An accuracy improvement of 3.08% compared to Sabrina et al. [13] and 1.01% compared to Ansori et al. [14], was obtained through a combination of normalized preprocessing with RobustScaler and comprehensive hyperparameter tuning. Fig. 3 compares the accuracy and F1-Score of the proposed LightGBM model against the baseline methods. The use of 5-Fold CV in this research provides more reliable performance estimation, because all data are used as the test set in rotation.

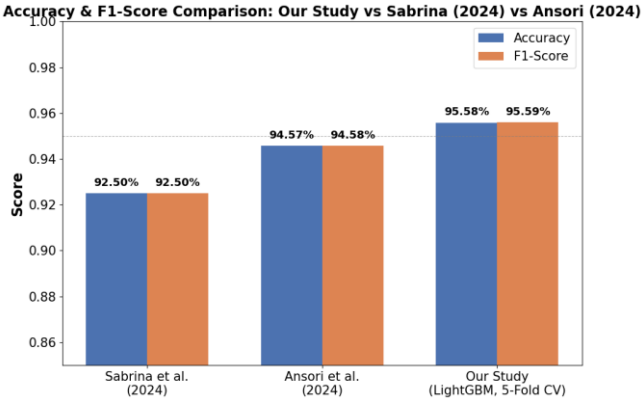


Fig. 3. Accuracy and F1-Score comparison between the proposed method and state-of-the -art studies

D. Analysis

The accuracy distribution each fold for LightGBM is,

- Fold 1 (95.62%)
- Fold 2 (96.18%)
- Fold 3 (95.66%)
- Fold 4 (94.75%)
- Fold 5 (95.70%)

The small variation between folds (maximum difference of 1.43%) confirms the consistency of the model. Fold 4 shows the lowest accuracy, this caused by a more difficult class distribution in that fold. Fig. 4 displays the confusion matrix of the optimized LightGBM model.

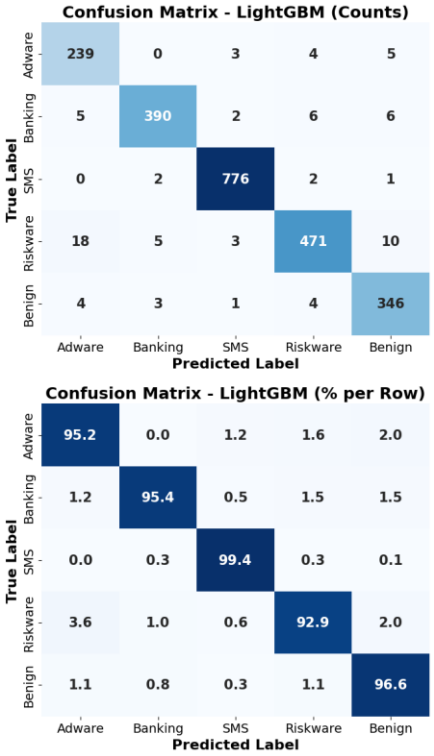


Fig. 4. Confusion matrix of the proposed LightGBM model on the test set

However, the model maintained a high baseline of performance even in the most difficult fold, underscores the effectiveness of the balanced training approach provided by in-fold SMOTE [15].

V. Conclusion

This study proposed an enhanced Android malware detection pipeline using LightGBM with optimized hyperparameters, Log1p and RobustScaler normalization, and in-fold SMOTE on the CICMalDroid2020 dataset. It is important to emphasize that the contribution of this work lies not in a novel algorithm, but in the systematic and empirically validated integration of established techniques into a single leakage-free pipeline. The research successfully addressed key methodological gaps identified in previous studies, particularly the risk of data leakage during oversampling and the limited reliability of single hold-out evaluation schemes. The research successfully addressed key methodological gaps identified in previous studies, particularly the risk of data leakage during oversampling and the limited reliability of single hold-out evaluation schemes.

The preprocessing stage revealed that 5,815 duplicate samples (33.5%) existed in the original dataset of 17.341 records, underscoring the importance of deduplication to prevent artificial performance inflation in the model training. Following deduplication, the combination of Log1p and RobustScaler normalization proved effective in reducing the influence of outliers inherent in statistical system call features, providing a more stable input representation for the classifier.

The experimental results demonstrated that LightGBM consistently outperformed all other models evaluated, including XGBoost, RandomForest, ExtraTrees, and four ensemble configuration based on hard voting. As shown in

Fig. 5, the analysis indicates that statistical features related to writev, mprotect, brk, futex, getpriority, ioctl, sigprocmask, write, pread64, and clock_gettime.

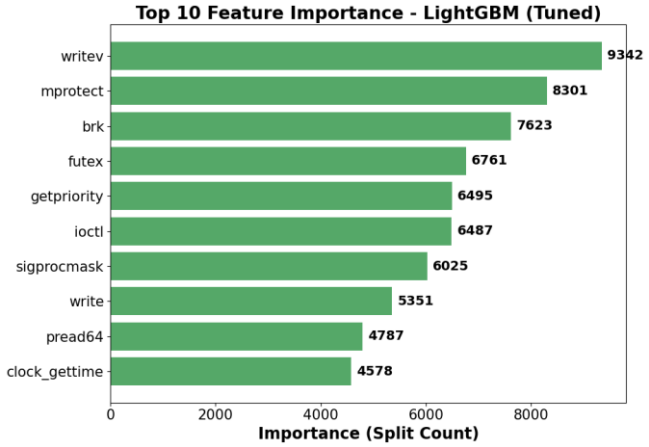


Fig. 5. Top 10 feature importance score extracted from the optimized LightGBM model

In the hold-out evaluation, LightGBM achieved the highest accuracy of 96.36%, while the 5-Fold Cross Validation confirmed a stable accuracy of 95.58% with a low standard deviation of ±0.0046. Both scores surpass the two State-Of-The-arts methods on the same dataset, namely Sabrina et al. [13] (92.50%) and Ansori et al. [14] (94.57%), representing improvements of 1.01% and 3.08% respectively.

Despite the robust performance of the proposed pipeline, this study has several limitations that should be acknowledged. First, the evaluation relies solely on the CICMalDroid2020 dataset, and the generalizability of the pipeline to other Android malware datasets or newer malware families has not been verified. Second, the model was trained and evaluated using only the 470 statistical features derived from dynamic system-call analysis, without incorporating complementary feature types such as static permissions, API call sequences, or network-traffic features. Third, the model's robustness against adversarial malware samples specifically crafted to evade detection, for example through feature perturbation or obfuscation, was not evaluated. Fourth, this study did not assess the model's ability to detect zero-day malware, i.e., malware families or variants absent from the training data, which limits conclusions about its performance against evolving real-world threats. Fifth, the feasibility of deploying the proposed pipeline in a real-time or resource-constrained mobile environment, including inference latency and computational overhead, was not investigated. While 5-Fold cross Validation ensures statistical reliability, it does not account for concept drift, where malware behaviors evolve over time. Future research will address these limitations along several directions. First, cross-dataset validation using additional Android malware datasets (e.g., CICAndMal2017, Drebin) will be conducted to assess the generalizability of the proposed pipeline. Second, robustness against adversarial and zero-day malware will be evaluated by incorporating adversarial training strategies and out-of-distribution detection techniques. Third, the feasibility of real-time, on-device deployment will be investigated including model compression and inference-latency benchmarking on representative mobile hardware. Finally, future work will integrate Deep Learning techniques, such as Graph Neural Networks (GNNs) or Long Short-Term Memory (LSTM) networks, to capture the sequential and structural relationships of API calls, potentially complementing the statistical feature-based approach used in this study.

REFERENCES

[1] A. Djenna, A. Bouridane, S. Rubab, and I. M. Marou, 'Artificial Intelligence-Based Malware Detection, Analysis, and Mitigation', *Symmetry*, vol. 15, no. 3, p. 677, Mar. 2023, doi: 10.3390/sym15030677.

[2] R. S. Arslan, 'Identify Type of Android Malware with Machine Learning Based Ensemble Model', in *2021 5th International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT)*, Ankara, Turkey: IEEE, Oct. 2021, pp. 628–632. doi: 10.1109/ISMSIT52890.2021.9604661.

[3] M. M.-H.-Z. Abedin and T. Mehrub, 'Comparison of Multiple Classifiers for Android Malware Detection with Emphasis on Feature Insights Using CICMalDroid 2020 Dataset', 2026, *arXiv*. doi: 10.48550/ARXIV.2602.00058.

[4] Nurul Qomariah, Erick Irawadi Alwi, and Muhammad Arfah Asis, 'Analisis Malware Hummingbad Dan Copycat Pada Android Menggunakan Metode Hybrid', *Cyber Secur. Dan Forensik Digit.*, vol. 6, no. 2, pp. 39–47, Feb. 2024, doi: 10.14421/csecurity.2023.6.2.4180.

[5] L. Marciano and A.-L. Camerini, 'Duration, frequency, and time distortion: Which is the best predictor of problematic smartphone use in adolescents? A trace data study', *PLOS ONE*, vol. 17, no. 2, p. e0263815, Feb. 2022, doi: 10.1371/journal.pone.0263815.

[6] P. Morán, A. Robles-Gómez, A. Duque, L. Tobarra, and R. Pastor-Vargas, 'Machine learning models and dimensionality reduction for improving the Android malware detection', *PeerJ Comput. Sci.*, vol. 10, p. e2616, Dec. 2024, doi: 10.7717/peerj-cs.2616.

[7] Ö. Kiraz and İ. A. Doğru, 'Visualising Static Features and Classifying Android Malware Using a Convolutional Neural Network Approach', *Appl. Sci.*, vol. 14, no. 11, p. 4772, May 2024, doi: 10.3390/app14114772.

[8] Sudhakar and S. Kumar, 'An emerging threat Fileless malware: a survey and research challenges', *Cybersecurity*, vol. 3, no. 1, p. 1, Dec. 2020, doi: 10.1186/s42400-019-0043-x.

[9] 'Malware Detection using Portable Executable Header and Gradient Boosting Classification Algorithm', in *Proceedings of the International Conference on Industrial Engineering and Operations Management*, Jahor Baru, Malaysia: IEOM Society International, Sep. 2022, pp. 916–924. doi: 10.46254/AP03.20220174.

[10] A. Mehrban and S. K. Geransayeh, 'RANSOMWARE THREAT MITIGATION THROUGH NETWORK TRAFFIC ANALYSIS AND MACHINE LEARNING TECHNIQUES'.

[11] C. Beck, A. Boumezoued, Y. Cherkaoui, E. Pradat, and B. Fleisher, 'Modelling financial losses from a ransomware attack using a causal approach', 2023.

[12] S. Sharma, Prachi, R. Chhikara, and K. Khanna, 'Dynamic analysis based Android Malware Detection

using ANN and RFE Feature Selection', in *2023 5th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*, Greater Noida, India: IEEE, Dec. 2023, pp. 1504–1509. doi: 10.1109/ICAC3N60023.2023.10541721.

[13] S. A. Amalina, F. A. Rafrastara, and M. Ibadullah, 'Android Malware Detection using Fast Correlation Based Filter and Random Forest Algorithm', in *2024 International Seminar on Application for Technology of Information and Communication (iSemantic)*, Semarang, Indonesia: IEEE, Sep. 2024, pp. 365–370. doi: 10.1109/iSemantic63362.2024.10762386.

[14] D. B. Ansori, J. Slamet, M. Z. Ghufron, M. A. R. Putra, and T. Ahmad, 'Android Malware Classification Using Gain Ratio and Ensembled Machine Learning', *Int. J. Saf. Secur. Eng.*, vol. 14, no. 1, pp. 259–266, Feb. 2024, doi: 10.18280/ijsse.140126.

[15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, 'SMOTE: Synthetic Minority Over-sampling Technique', *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002, doi: 10.1613/jair.953.

[16] S. Stocksieker, D. Pommeret, and A. Charpentier, 'Generalized Oversampling for Learning from Imbalanced datasets and Associated Theory: Application in Regression'.

[17] Y. Zhao *et al.*, 'On the Impact of Sample Duplication in Machine-Learning-Based Android Malware Detection', *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 3, pp. 1–38, Jul. 2021, doi: 10.1145/3446905.

[18] Y. Mu, M. Jin, X. Song, and N. Aletras, 'Enhancing Data Quality through Simple De-duplication: Navigating Responsible Computational Social Science Research', in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 12477–12492. doi: 10.18653/v1/2024.emnlp-main.694.

[19] F. Alserhani and A. Aljared, 'Evaluating Ensemble Learning Mechanisms for Predicting Advanced Cyber Attacks', *Appl. Sci.*, vol. 13, no. 24, p. 13310, Dec. 2023, doi: 10.3390/app132413310.

[20] S. Mishra, N. S. Alshammari, H. Hussain, and R. A. Alfahidah, 'A cross-dataset harmonized intrusion detection framework with statistically validated multi-model learning', *PLOS One*, vol. 21, no. 4, p. e0346982, Apr. 2026, doi: 10.1371/journal.pone.0346982.

[21] K. J. Mwiga, M. A. Dida, A. Mohsin, and I. H. Sarker, 'A Generative AI Method for Minority Class Handling in Anomaly Detection with Drift and Explainability Analysis', *New Gener. Comput.*, vol. 44, no. 2, p. 13, May 2026, doi: 10.1007/s00354-026-00318-8.

[22] A. Rehman, T. Saba, M. M. Jamjoom, S. Al-Otaibi, and M. I. Khan, 'Advances in Machine Learning for Explainable Intrusion Detection Using Imbalance Datasets in Cybersecurity with Harris Hawks Optimization', *Comput. Mater. Contin.*, vol. 86, no. 1, pp. 1–15, 2026, doi: 10.32604/cmc.2025.068958.

[23] H. Semmelrock *et al.*, 'Reproducibility in machine-learning-based research: Overview, barriers, and drivers', *AI Mag.*, vol. 46, no. 2, p. e70002, Jun. 2025, doi: 10.1002/aaai.70002.

[24] A. Verma, 'An Optimized LightGBM Framework for Static Malware Classification Based on Windows Portable Executable File Attributes', *Int. J. Comput. Model. Appl.*, vol. 2, no. 4, pp. 32–40, Jan. 2026, doi: 10.63503/j.ijcma.2025.208.

[25] Y. Hu, K. Xiao, L. Luo, and L. Chen, 'An XGBoost-Based Intrusion Detection Framework with Interpretability Analysis for IoT Networks', *Appl. Sci.*, vol. 16, no. 2, p. 980, Jan. 2026, doi: 10.3390/app16020980.

[26] I. Bibers, O. Arreche, W. Alayed, and M. Abdallah, 'Ensemble-IDS: An Ensemble Learning Framework for Enhancing AI-Based Network Intrusion Detection Tasks', *Appl. Sci.*, vol. 15, no. 19, p. 10579, Sep. 2025, doi: 10.3390/app151910579.

[27] S. Arlot and A. Celisse, 'A survey of cross-validation procedures for model selection', *Stat. Surv.*, vol. 4, no. none, Jan. 2010, doi: 10.1214/09-SS054.

[28] M. Owusu-Adjei, J. Ben Hayfron-Acquah, T. Frimpong, and G. Abdul-Salaam, 'Imbalanced class distribution and performance evaluation metrics: A systematic review of prediction accuracy for determining model performance in healthcare systems', *PLOS Digit. Health*, vol. 2, no. 11, p. e0000290, Nov. 2023, doi: 10.1371/journal.pdig.0000290.